

Drawbacks before using (VCS)

- > As modifications were done in the same file , so any changes made at any particular instant , tracking them was difficult.
- > Collaboration was not there
- > Any damage to the machine , might lead to loss of source code .
- > May lead to lots of rework and eventually delay in projects .

To overcome with these problems , VCS(Version Control System) came to existence .

VCS

vcs is used to manage and track the versions of the Source Code .
each time the file is being modified , it will be treated as a " version "

- + " manage " - means we can go to any of the versions & if needed we can do the modifications as well .
- + " track " - Who -> who made the changes
when -> When the changes were made
what -> What changes/modifications have been made
why -> Why we made these changes

Types of VCS :

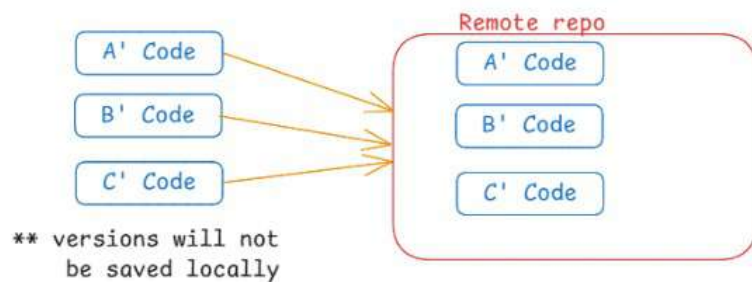
i> Local/Localized VCS - Save the versions of the code Separately on the local machine

eg .. RCS (Revision Control System)
SCCS (Source Code Control System)

Drawbacks :

- > Any physical damage to machine may lead to loss of Source Code
- > If Source Code is lost , It will lead to lots of rework
- > No Collaboration

ii> Central VCS - In CVCS , the versions will not be saved locally but instead , they will be stored or uploaded in a remote Server .



eg .. SubVersion
Microsoft team Foundation Server

- > Saving the versions of the source code directly to the remote repository
- > Collaboration is there ...

Drawbacks :

- > If there is no stable internet connection , then we cant save the versions in a remote server...
- > if any problem occurs to the server ..
(physical damage , Server downtime etc)

iii> Distributed VCS - It can be defined as the combination of both local Vcs and Central Vcs .
Here we will be saving the versions both locally as well as in the remote server .

eg .. GIT
Mercurial

- > Git is used to manage and track the changes made in the Source code
- > GitHub is a Platform that hosts Git Repositories

History of Git

- > Linus Torvald introduced git in 2005 , which was totally free to use and was a open source Software .
- > earlier people were using a tool called "Bitkeeper"

Features of Git

- > Free and Open Source
- > Used to track changes in Source Code
- > Supports Collaboration
- > Certain Operations such as branching , merging are way more faster in comparison to other Version Control Tool

Installing

```
sudo apt update && sudo apt install git -y
```

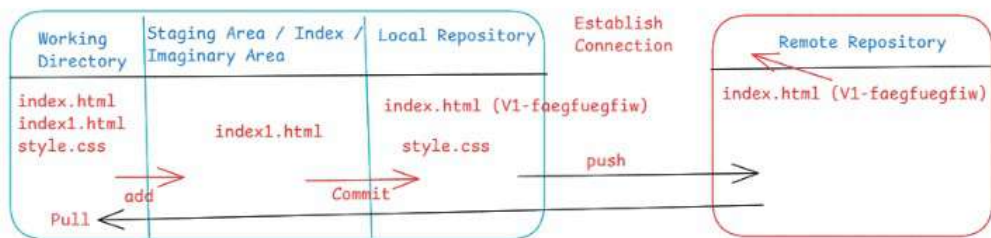
Checking

```
git --version
which git
```

Working Stages of Git



```
index.html
index1.html
styles.css
```



- Working Directory - Working Directory is a place where we will be Working / Creating / Modifying the files or perform any kind of read/write operations
- Staging Area/Index - Its a place where the files will be kept for a temporary period of time
- Local repository - Its a place where we are going to save files/versions of code permanently in local Machine.

Workflow of Git

1. Initialization - To get started using git for any resource , we need to initialize the folder/directory .
Once after initializing , we will be getting a hidden folder ".git " & also the folder structure will get segregated in 3 distinct sections .

```
git init
```
2. Configuration - Username & User Email will be configured here which will be later used to provide info for who has made the changes

```
git config user.name "Rahul"
git config user.email "Rahul@gmail.com" // Applied to only the specific repo

git config --global user.name "Rahul" // Applied to all repos for the user
git config --global user.email "Rahul@gmail.com"

git config user.name
git config user.email // To check the Configured Credentials
```

```
>> Create/Write/modify all the Source codes here
touch file.txt
nano file.txt
```

3. Add - To COPY the files from the Working Directory to the Staging Area for a temporary period of time.

```
git add filename.extension
-> git add index.html // Add single file

git add filename filename filename
-> git add a.txt b.txt c.txt // Add Multiple Files

git add .
-> git add . // Add all files in my current Directory

git add *.extension // To Add all files of a particular extension
-> git add *.java
```

>> To check the files which are present in the staging/Index area : `git status`

** Tracked files are shown in green
** Untracked files are shown in Red

4. Commit - MOVING files from the index to the Local Repository and Saving it Permanently.

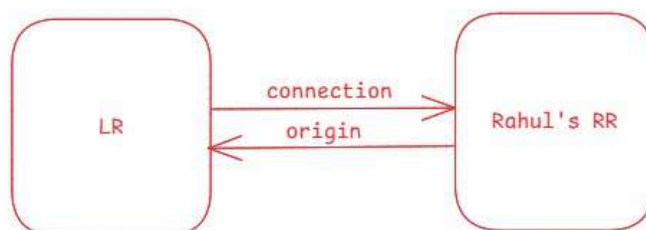
```
git commit -m "Any message"
-> git commit -m "Committing index files" index.html index1.html
```

>> To check the Commit history : `git log`
`git log --oneline` //shows one line per commit (clean view)
`git log --graph` //shows visual branch structure

commit5
commit4
commit3
commit2
commit1

5. Establish Connection - In order to upload the changes from the Local repository to the remote Repository (GitHub Repo)

```
git remote add alias_name remote_url
git remote add origin <Remote_Repository_link>
```



// Here alias is nothing but the alternate name/nickname we are giving to the remote repository.

>> To Verify the Connection: `git remote -v`

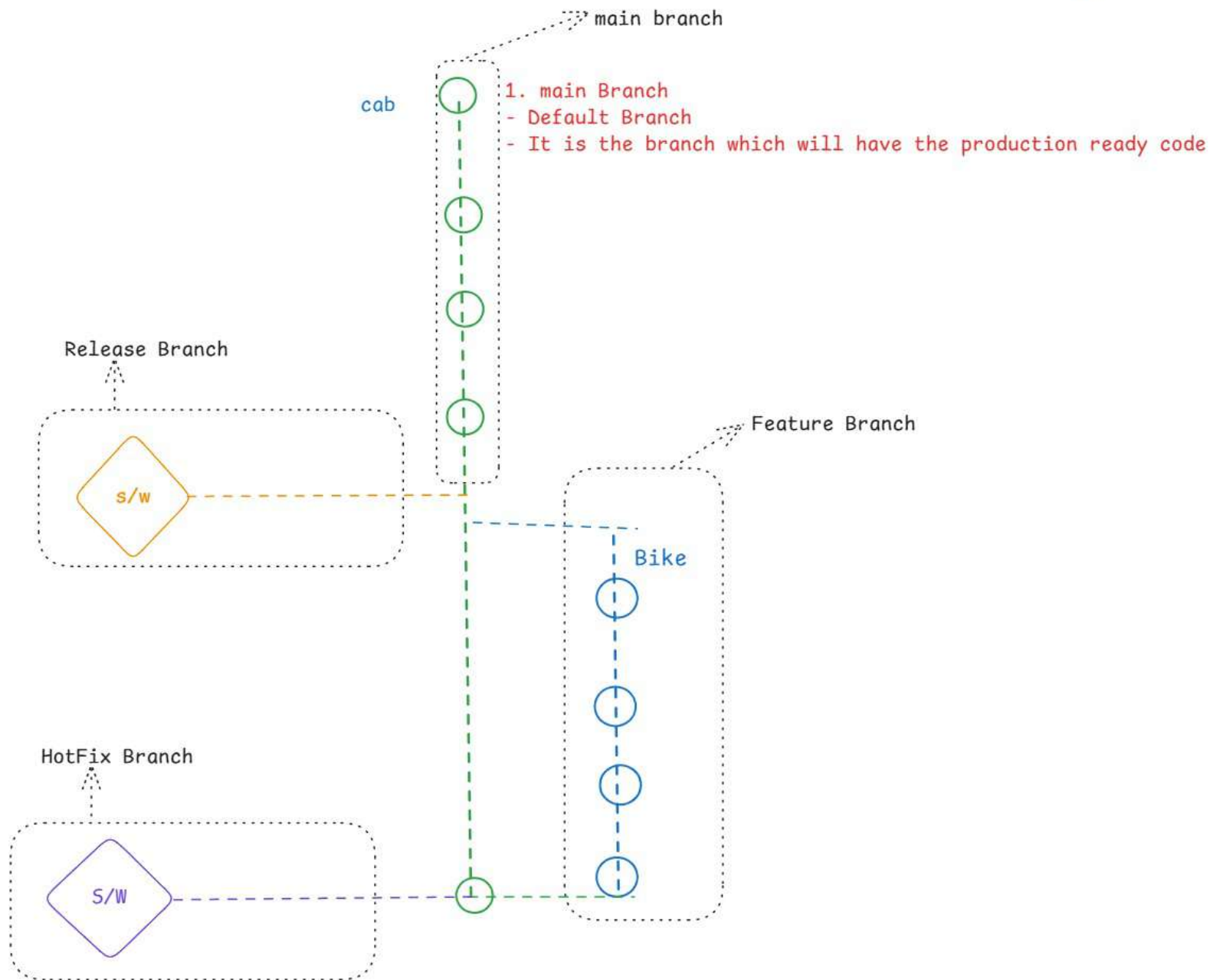
>> To remove the Connection : `git remote remove alias_name`
`git remote remove origin`

6. Push : Uploading the files/changes in the remote repository

```
git push origin master/main
alias_name <--> branch you want to push
```

6. Pull : To get the changes from the remote repository to the working directory

```
git pull origin master/main
alias_name <--> branch you want to pull
```



Types of Branches

- > Master/ Main Branch - Default Branch
It will have the stable production ready code
- > Release Branch - Branch where we will prepare for releasing
- > Feature Branch - Branch which is used to introduce the new features .
- > HotFix Branch - Fix immediate issues and put it back for use

Benefits of using Branching

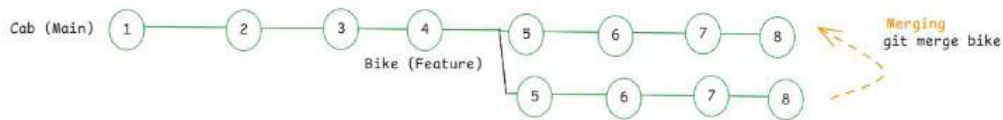
- > Provides isolation and independent line of development
- > It also provides better collaboration b/w the teams as they can work on multiple features simultaneously .
- > If anything goes wrong with any branch , we can simply delete it without affecting the main branch.

Commands in Branching

- > To list the branches present locally : `git branch`
`git branch --list`
 - > To Create a Branch : `git branch <branch_name>`
- `git branch auto`
 - > To Switch to a branch : `git switch branch_name`
`git checkout branch_name`
 - > To create and switch simultaneously to a particular branch : `git checkout -b branch_name`
 - > To push any branch to remote repo : `git push <origin> <branch_name>`
`git push origin cab`
 - > To pull any branch from remote repo : `git pull <origin> <branch_name>`
`git pull origin auto`
 - > To list all the branches present remotely : `git branch -r`
 - > To list all the branches present remotely as well as locally : `git branch -a`
 - > To delete the branches locally : `git branch -d branch_name` (Safe Delete)
 - Delete those branch which are already added/merged to the current branch`git branch -D branch_name` (Force Delete)
 - Delete those branch which are not already added/merged to the current branch
 - > To Delete the branches remotely : `git push origin -d branch_name`
 - > To rename the branch : `git branch -m old_branch_name new_branch_name`
- # HEAD is a pointer , which points to the tip of the current branch or to the latest commit of the current branch.

Merging

> Combining the changes from one branch to another branch



> The command for merging branches : `git merge <branch_name>`

```
merge branch1 with branch2
## we need to be present in branch2
## the command goes like "git merge branch1"
```

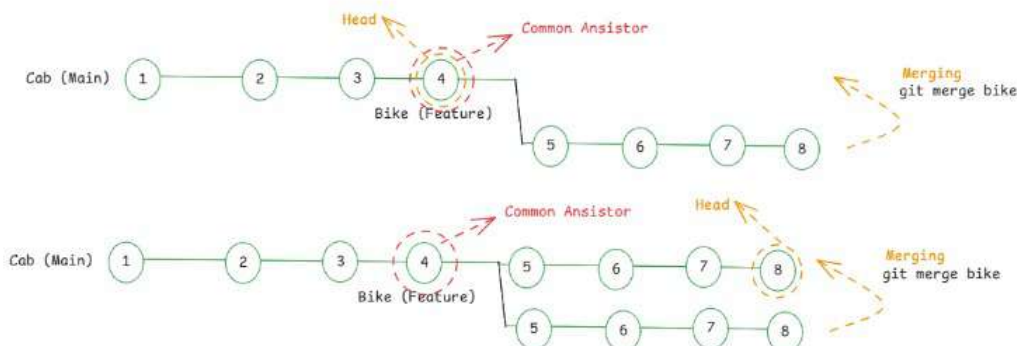
> There are two types of merging :

1. Two-way merging / Fast Forward Merging
2. Three-way merging / ORT strategy merging (Optimised Recursive Three-way)

Two way merging / Fast Forward Merging

- > If no further commits were made in the master branch after creation of the new branch , then the merging will be treated as Two way Merging / Fast Forward Merging.

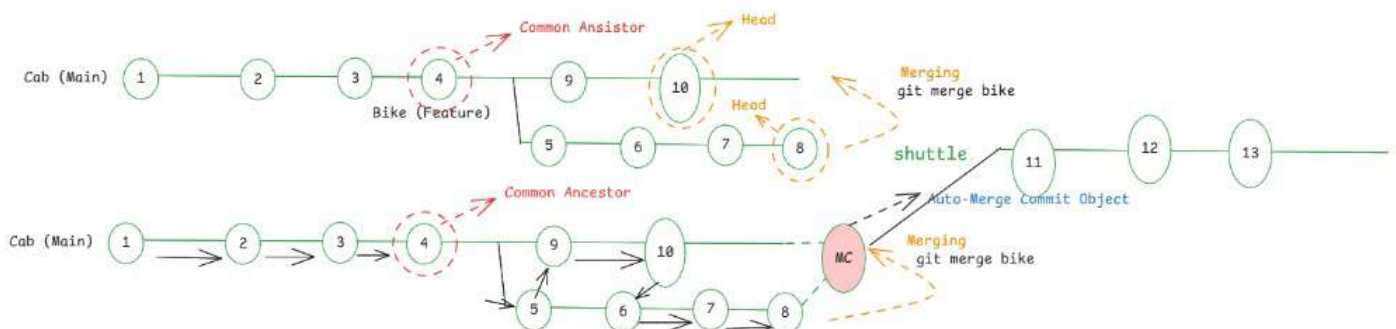
head - It is a pointer which points to the latest commit of the current branch .
base / Common ancestor - The latest Commit which is present in both the branches .



>> While 2 way merging , it just considers two things
->The head of the Current Branch
->The common Ancestor

Three way merging / ORT strategy

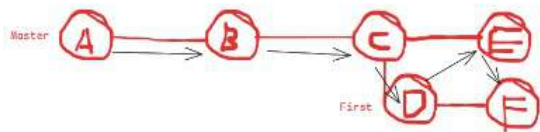
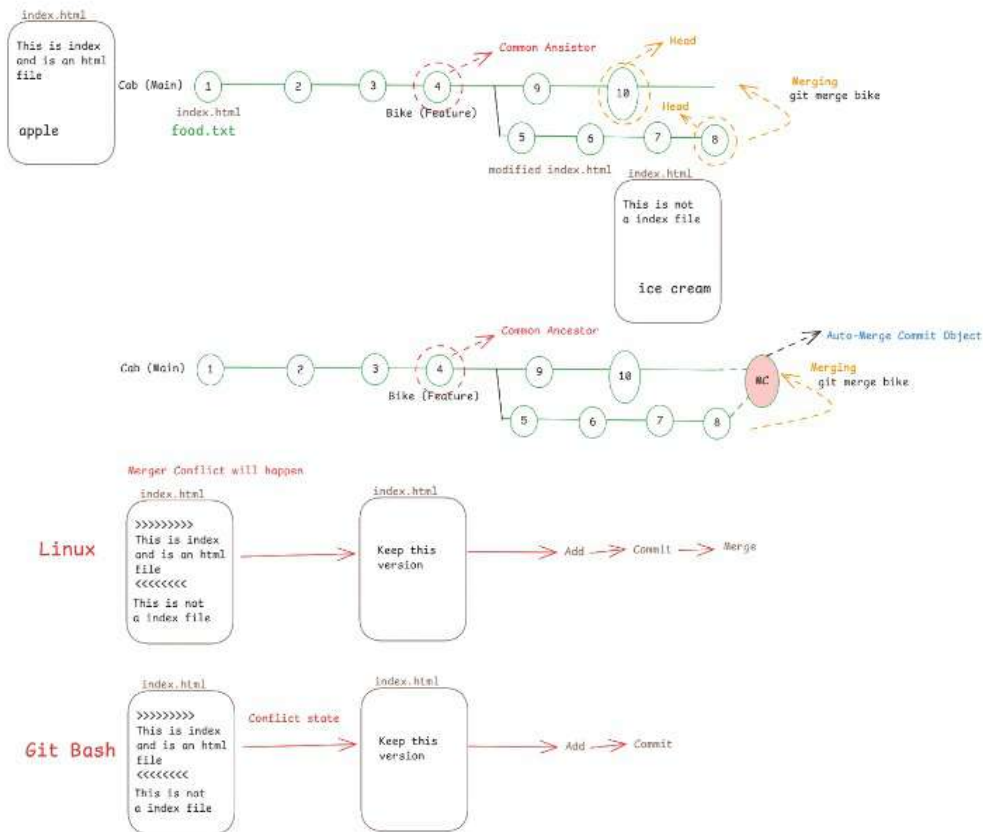
- > If there are further commits made after the creation of the bike branch , then the merging will be treated as 3 way merging or ORT strategy merging.



>> While 3 way merging , it just considers three things
->The head of the Current Branch
->The common Ancestor
->The Head of the Merging Branch

Merge Conflict

- > Merge conflict is a phenomenon that will happen when we will try to merge the same file (which might get edited) from different branch .

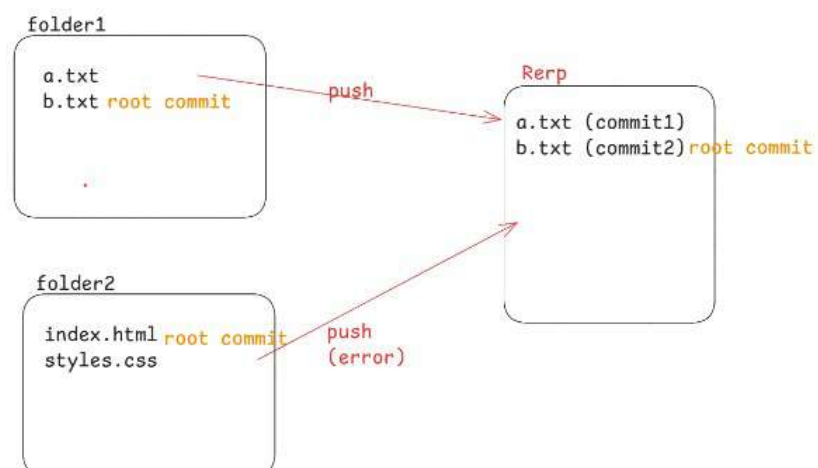
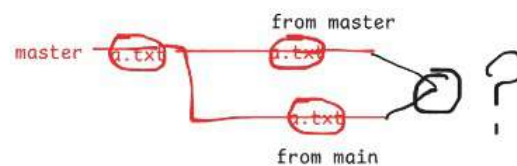


Type -> Three way Merging

Order of gitlog after merging

MG -> F -> E -> D -> C -> B -> A

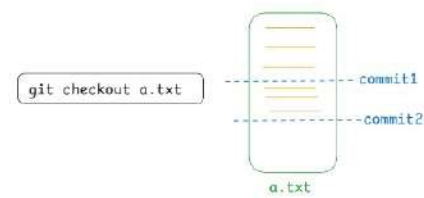
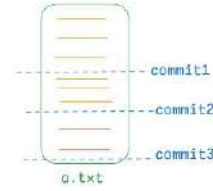
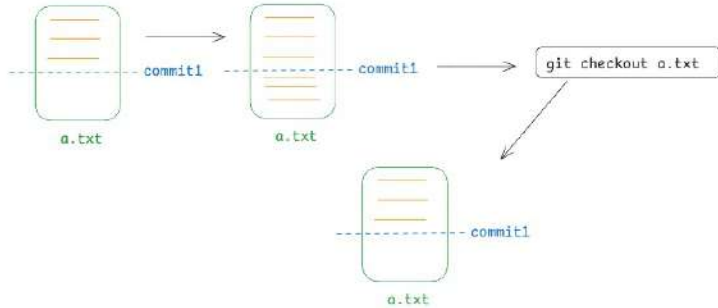
Number of commits in master branch -
=> 7



Undoing Changes

Checkout :

>> To undo the changes in the working Directory we use checkout command .



>> " git checkout filename " will match the content of the file with the last commit's content .

>> Undo the modifications for a particular file : git checkout <file_name>

>> Undo the modifications for a particular file with respect to the specified commit : git checkout <commit_id> <file_name>

>> Undo the modifications for all the files : git checkout .

Reset :

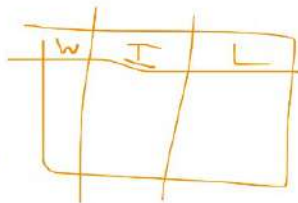
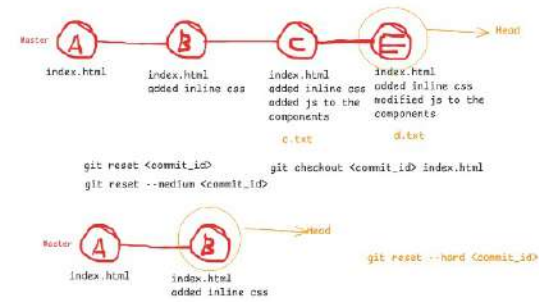
>> If we need to undo the changes in the staged files , we can use the reset command .

>> Whenever we are using reset command , it will move the files from the staging area back to the working Directory

>> Unstaging a particular file : git reset file_name

>> Unstaging all the files : git reset .

>> Flags to be used along with reset command :



--soft

- > Moves HEAD to the specified commit
- > Keeps the changes staged (in tracking stage)
- > Working directory remains unchanged
- > useful for redoing a commit with new changes
- > git reset --soft <commit_id>

--mixed

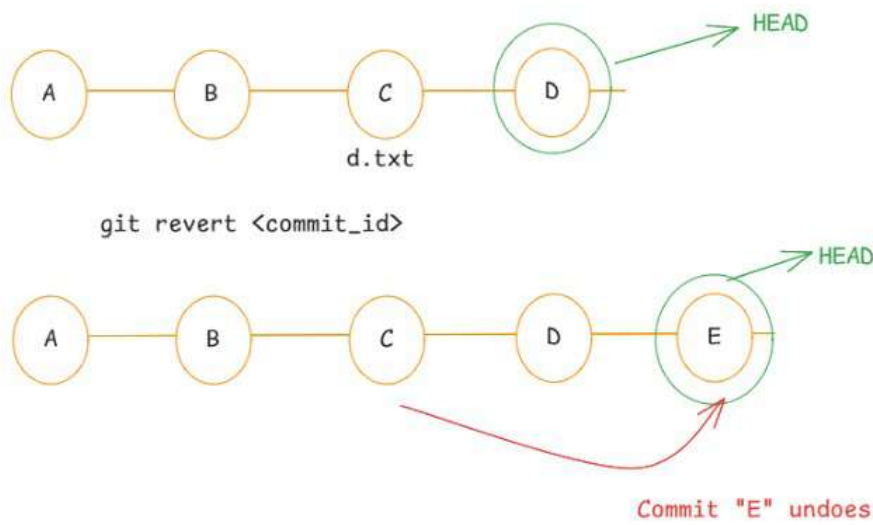
- > Moves HEAD to the specified commit
- > Unstage the changes (in non - tracking stage)
- > Working directory remains modified
- > useful for un-committing changes but keeping them as unstaged
- > git reset --mixed <commit_id>

--hard

- > Moves HEAD to the specified commit
- > Discards all staged and working directory changes (irreversible)
- > use with caution - this deletes uncommitted changes
- > git reset --hard <commit_id>

Revert :

- >> git revert is used to undo changes from a previous commit by creating a new commit that reverses or Inverses the effects of the target commit. unlike git reset (Which re -writes the history) , revert is non - destructive and safe for repositories



Cloning :

- >> Git Clone is used to create a local copy of a remote git repository . It downloads the entire project , including all files , branches and commit histories.

```
git clone <remote_url>
git clone <remote_url> destination_folder
```

refers to the directory where the repository will be saved

Optional

Fetching :

- >> The git fetch command retrieves the latest changes from the remote repository , without modifying our working Directory . It helps us to inspect updates before integrating them with git merge or git rebase .

```
git fetch remote_server remote_branch
e.g.. git fetch origin master
```

- >> Fetch + Merge = Pull

Forking :

- >> Getting a copy of another repo in your github account .
 - Used to Contribute to open source Projects without direct access.

Readme File :

- >> A text file which contains information about the project for users and developers like (name of the project , purpose of the project , any guidelines etc)

.gitignore :

- >> It is a file where we can mention those files/directories that needs to be ignored , i.e. they wont be tracked/committed to the repo

Commands in Branching

> To list the branches present locally : `git branch`
`git branch --list`

> To Create a Branch : `git branch <branch_name>`
- `git branch auto`

> To Switch to a branch : `git switch branch_name`
`git checkout branch_name`

> To create and switch simultaneously to a particular branch : `git checkout -b branch_name`

> To push any branch to remote repo : `git push <origin> <branch_name>`
`git push origin cab`

> To pull any branch from remote repo : `git pull <origin> <branch_name>`
`git pull origin auto`

> To list all the branches present remotely : `git branch -r`

> To list all the branches present remotely as well as locally : `git branch -a`

> To delete the branches locally : `git branch -d branch_name` (Safe Delete)
- Delete those branch which are already added/merged to the current branch
`git branch -D branch_name` (Force Delete)
- Delete those branch which are not already added/merged to the current branch

> To Delete the branches remotely : `git push origin -d branch_name`

> To rename the branch : `git branch -m old_branch_name new_branch_name`

HEAD is a pointer , which points to the tip of the current branch or to the latest commit of the current branch.